Main method

Creating a New Class with a Main Method in Java

To create a new class in Java and include a `main` method, follow these steps:

1. **Define the Class**: The class is created using the `class` keyword, followed by the class name. By convention, class names begin with an uppercase letter.
2. **Create the Main Method**: The main method is the entry point of any Java program. It is defined with the signature `public static void main(String[] args)`. This method is executed when the class is run.

Code Example:

```
// Define the class named 'Greeting'
public class Greeting {

    // Main method - entry point of the program
    public static void main(String[] args) {

        // Print a greeting message to the console
        System.out.println("Hello, welcome to the Java program!");

        // Create an object of the Greeting class
        Greeting greet = new Greeting();

        // Call the custom method on the created object
        greet.sayHello();
    }

    // A custom method to say hello
    public void sayHello() {
        System.out.println("Hello from the Greeting class!");
    }
}
```

Explanation:

1. **Class Definition**: The class is named `Greeting`, and it is declared with the `public` keyword so it can be accessed from outside the class.
2. **Main Method**: The `main` method is the entry point of the program. Inside the main method:
    - It prints a greeting message to the console using `System.out.println()`.

- It creates an object `greet` of the `Greeting` class.
- It calls the `sayHello()` method on that object to print another message.

3. **Custom Method**: The `sayHello()` method is a simple instance method that prints a message when invoked.

Output:

```
Hello, welcome to the Java program!
Hello from the Greeting class!
```

How to Run the Program:

1. Save the code in a file named `Greeting.java`.
2. Compile the file using the command:

   ```
   javac Greeting.java
   ```

3. Run the compiled class using the command:

   ```
   java Greeting
   ```

This will execute the `main` method, and you will see the greeting messages printed on the screen.

The `main` Method in Java

The `main` method is the entry point for any standalone Java application. When you run a Java program, the Java Virtual Machine (JVM) looks for the `main` method to start the execution of the program.

Syntax of the `main` Method:

```
public static void main(String[] args)
```

Each keyword and component in this method has a specific purpose.

1. `public`:
   - This is an **access modifier**. It means that the `main` method is accessible from anywhere.

- For Java to execute the `main` method, it needs to be accessible to the JVM, which is why it is declared as `public`.

2. `static`:
   - The `static` keyword means that the method belongs to the class itself, not to instances (objects) of the class. This is important because the JVM does not create an instance of the class before calling the `main` method—it calls it directly on the class.
   - Without `static`, the JVM would have to create an object of the class before calling the `main` method, which is not the case.
3. `void`:
   - `void` means that the `main` method does not return any value. The `main` method is responsible for starting the execution of the program, and it does not need to send any result back to the JVM.
4. `main`:
   - This is the name of the method. The JVM looks for a method named `main` to start the program.
5. `String[] args`:
   - This is a parameter of type `String[]` (an array of strings).
   - It allows the user to pass command-line arguments to the program when it is executed. These arguments are stored in this array, and the program can access them for various purposes.
   - For example, if you run `java MyProgram arg1 arg2`, the `args` array will contain `["arg1", "arg2"]`.

Working of the `main` Method:

- **Execution Flow**: When you run a Java program, the JVM searches for the `main` method in the class you're running. Once found, the JVM invokes the `main` method and starts executing the code inside it.
- **Single Entry Point**: You can only have one `main` method in a class when executing a Java program. However, a class can have multiple methods with the name `main` if they have different parameters (method overloading), but only the `main(String[] args)` version is used to start the program.

Example:

Here’s a simple example to demonstrate how the `main` method works:

```java
public class MainExample {

    // Main method - entry point of the program
    public static void main(String[] args) {

        // Print a simple message to the console
        System.out.println("Hello, this is the main method!");

        // Check if any command-line arguments were passed
        if (args.length > 0) {
            System.out.println("Command-line arguments:");
            for (String arg : args) {
                System.out.println(arg);
            }
        } else {
            System.out.println("No command-line arguments were passed.");
        }
    }
}
```

Explanation:

1. **Main Method**: The `main` method is where the program begins its execution.
2. **Command-Line Arguments**: If you run the program from the command line with arguments, they are passed to the `main` method through the `args` array. For example:

   ```
   java MainExample Hello World
   ```

   The `args` array will contain `["Hello", "World"]`, and the program will print these values.

How to Run the Example:

1. Save the code in a file named `MainExample.java`.
2. Compile it using the following command:

   ```
   javac MainExample.java
   ```

3. Run the program:

```
java MainExample Hello World
```

Output:

```
Hello, this is the main method!
Command-line arguments:
Hello
World
```

If you run the program without arguments:

```
java MainExample
```

Output:

```
Hello, this is the main method!
No command-line arguments were passed.
```

Important Points About the Main Method:

1. **Entry Point**: The `main` method is the entry point for any standalone Java application. The JVM uses it to start the program.
2. **No Return**: The `main` method is always `void`, meaning it does not return any value to the JVM.
3. **Static**: Since the `main` method is static, it can be executed without creating an object of the class.
4. **Arguments**: The `args` parameter allows command-line arguments to be passed to the program when it is run.
5. **Only One Main Method**: There can be only one `main` method in a class for execution. However, a class can have multiple methods with the same name (`main`), as long as their parameters are different (method overloading), but only the one with `String[] args` is used by the JVM for starting the program.
